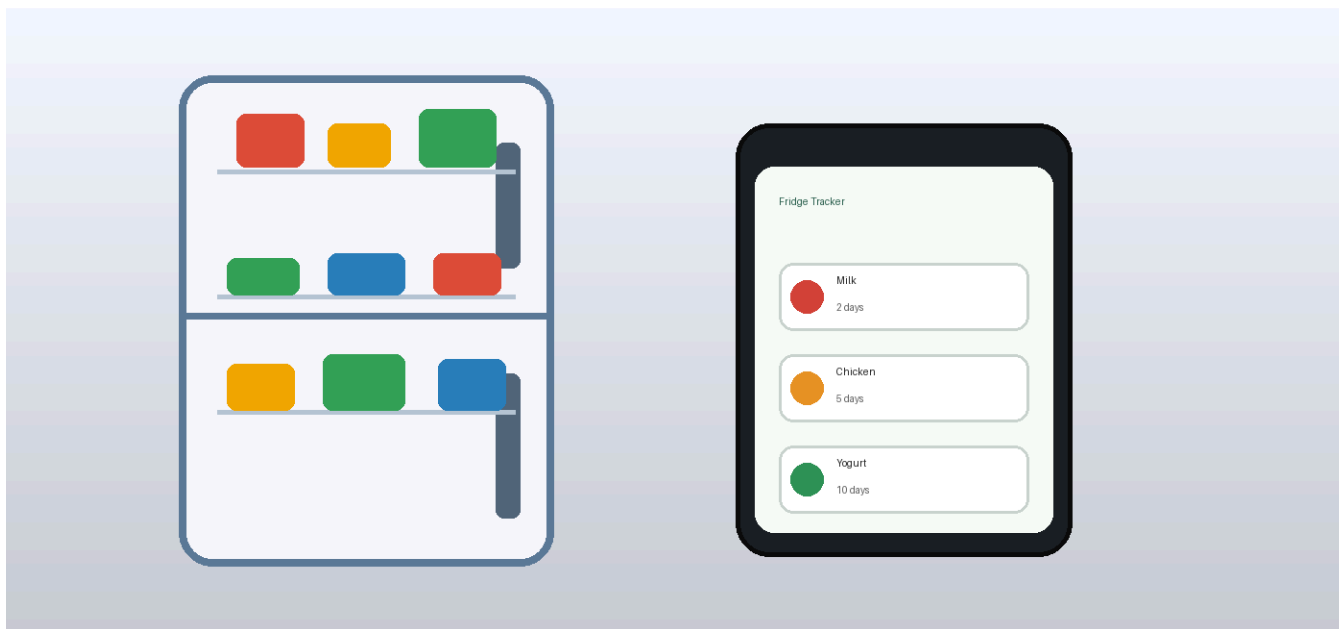




Fridge Tracker

Svendeprøve til Datatekniker med Speciale i Programmering

af Andrei, Azad og Dylan

Technical Education Copenhagen, Ballerup

Kravspecifikation

Vejleder: Henrik

Afleveringsfrist for kravspecifikationen: 15.06.2026

1 INDLEDNING.....	3
1.1 Projektgruppen.....	3
2 BAGGRUND.....	4
2.1 Problemformulering.....	4
2.2 Gevinster ved løsningen.....	4
2.3 Løsningen set fra brugerens side.....	4
2.4 Omfang og afgrænsninger.....	4
2.5 Fremtidige løsninger.....	4
3 GENEREL BESKRIVELSE.....	5
3.1 Løsningsskitse.....	5
3.2 Aktør-kontekstdiagram.....	7
3.3 Aktørbeskrivelse.....	8
3.4 Use case-diagram.....	9
3.5 Hardware/software-miljø.....	10
3.6 Sikkerhed.....	11
3.7 Testsituationer og acceptkriterier.....	11
4 FUNKTIONELLE KRAV.....	12
4.1 Use Cases.....	12
4.1.1 Opret konto og log ind.....	12
4.1.2 Se køleskabsoversigt.....	14
4.1.3 Tilføj vare manuelt.....	15
4.1.4 Scan og tilføj vare.....	16
4.1.5 Få opskriftsforslag.....	17
4.1.6 Slet og gendan vare.....	18
4.1.7 Brug delt køleskab.....	19
4.1.8 Udløbsnotifikation - senere version.....	20
4.2 Krav om funktionalitet.....	21
5 IKKE-FUNKTIONELLE KRAV.....	22
6 DESIGNKRAV.....	23
6.1 Brugerfladekrav.....	23
6.2 Visuelt flow.....	23
7 DOKUMENTHISTORIK.....	24
8 BILAG.....	25
8.1 Definitioner og ordforklaringer.....	25
8.2 Datamodel.....	26

1 INDLEDNING

Dette dokument definerer kravene til udviklingen af Fridge Tracker. Projektet er en mobilapplikation, som hjælper brugere med at holde overblik over varer i køleskabet, se udløbsdatoer og vælge de varer, der bør bruges først.

Idéen opstod ud fra et ønske om at reducere madspild og gøre registrering af madvarer hurtigere gennem manuel tilføjelse, kamerascanning, stregekodeopslag og automatiske forslag til udløbsdato.

1.1 Projektgruppen

- Andrei
- Azad
- Dylan

2 BAGGRUND

2.1 Problemformulering

Mange husholdninger mangler et præcist overblik over, hvilke varer de har i køleskabet, og hvornår varerne udløber. Det kan føre til dobbeltindkøb, glemte madvarer og unødigt madspild.

Eksisterende løsninger kræver ofte meget manuel registrering. Projektet fokuserer derfor på følgende spørgsmål:

1. Hvordan kan en cross-platform mobilapp gøre det hurtigt og intuitivt at holde styr på køleskabets indhold?
2. Hvordan kan scanning, standardudløbsdatoer, farvekoder og opskriftsforslag hjælpe brugeren med at reducere madspild?

2.2 Gevinster ved løsningen

- Bedre overblik over varer og udløbsdatoer.
- Mindre risiko for dobbeltindkøb.
- Tydelig prioritering med rød, orange og grøn udløbsstatus.
- Hurtigere registrering med kamera og strejkode.
- Opskriftsforslag baseret på køleskabets varer.
- Mulighed for fælles køleskab i en husstand.

2.3 Løsningen set fra brugerens side

Brugeren opretter en konto eller logger ind og får adgang til sin køleskabsoversigt. Varerne vises med navn, kategori og udløbsdato. Udløbsstatus vises med rød markering ved 3 dage eller mindre, orange ved 4-7 dage og grøn ved mere end 7 dage.

Brugeren kan tilføje varer manuelt eller anvende kameraet til tekstgenkendelse og strejkodescanning. Appen kan hente produktdata fra Open Food Facts og foreslå en standardudløbsdato fra en lokal JSON-database.

2.4 Omfang og afgrænsninger

Den nuværende version omfatter login, køleskabsoversigt, manuel tilføjelse, scanning, automatisk standardudløb, sletning med fortryd, opskriftsforslag og delt køleskab.

Udløbsnotifikationer er planlagt til en senere version og indgår ikke som et krav til godkendelse af den nuværende version. Open Food Facts og opskriftsfunktioner kræver internetforbindelse.

2.5 Fremtidige løsninger

- Udløbsnotifikationer på telefonen.
- Direkte navigation fra en notifikation til en bestemt vare.
- Madspildsstatistik baseret på brugte og smidte varer.

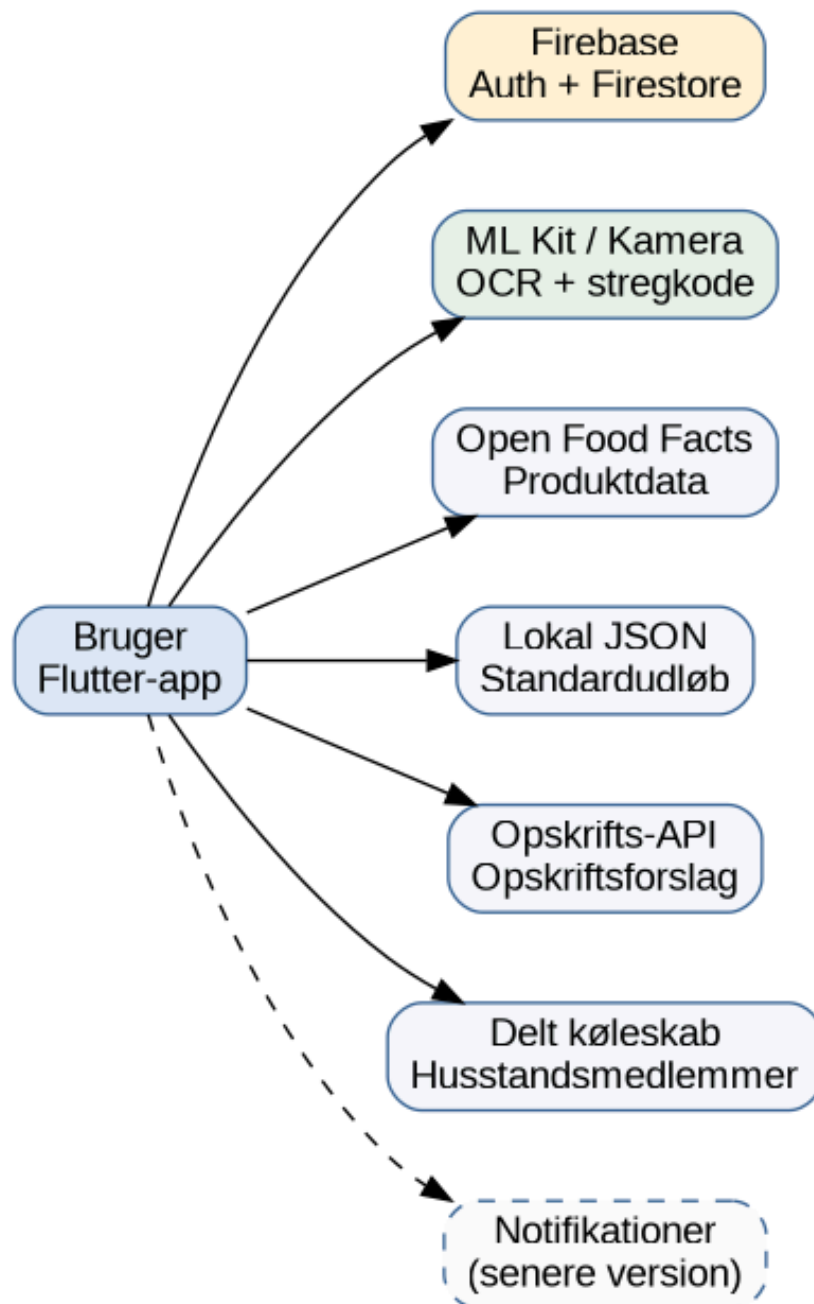
- Flere sprog og bedre tilgængelighedsindstillinger.

3 GENEREL BESKRIVELSE

3.1 Løsningsskitse

Fridge Tracker består af en Flutter-app, Firebase Authentication, Cloud Firestore, lokale standardudløbsdata samt eksterne produkt- og opskrifts-API'er. En oversigt over løsningen fremgår af figur 1.

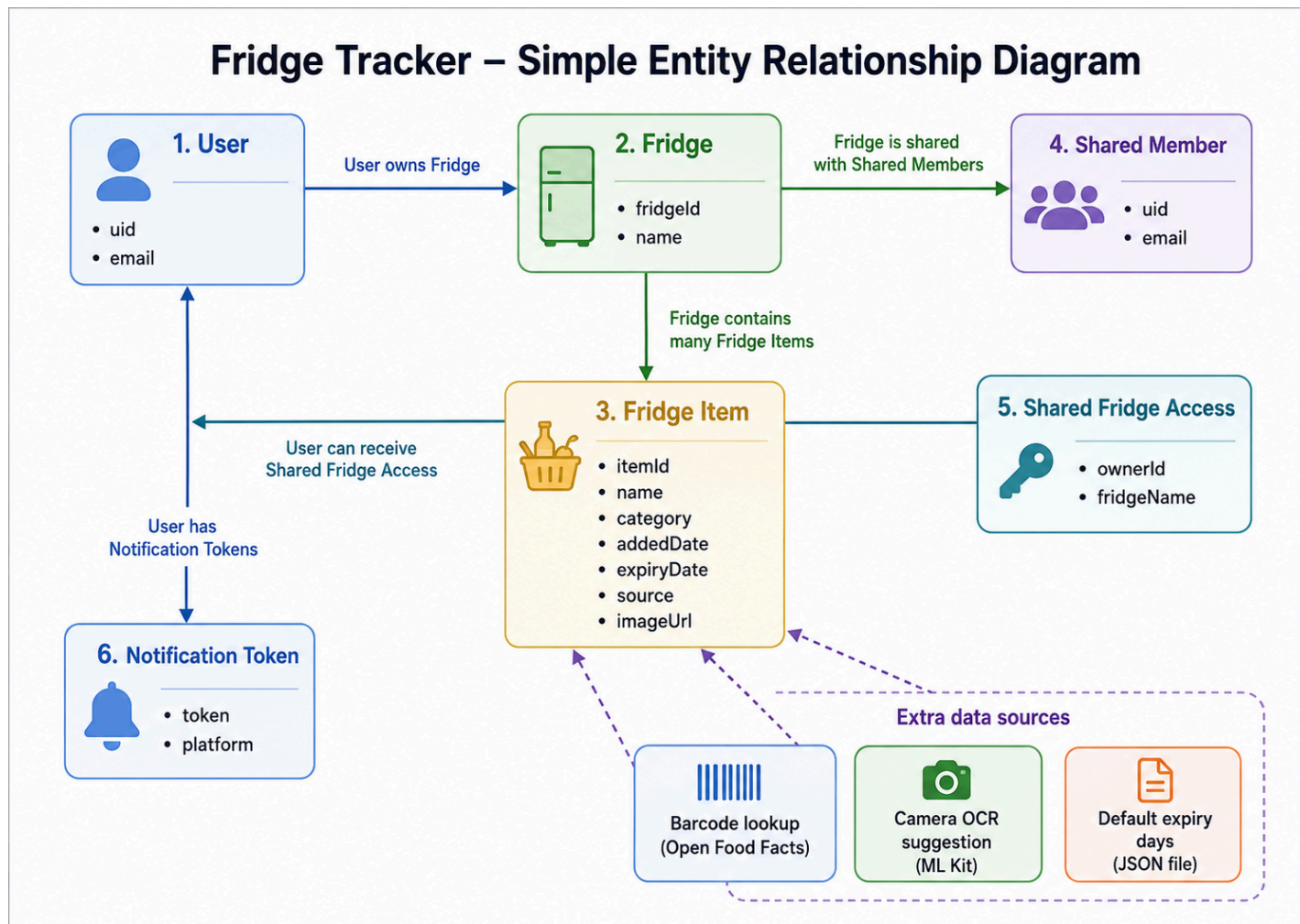
Figur 1 - Løsningsskitse



3.2 Aktør-kontekstdiagram

Figur 2 viser systemet, dets brugere og de eksterne systemer, som Fridge Tracker kommunikerer med. En aktør kan være en personrolle eller et eksternt system, der sender data til eller modtager data fra appen.

Figur 2 - Aktør-kontekstdiagram



3.3 Aktørbeskrivelse

Aktør Navn:	Bruger
Type:	Primær aktør
Beskrivelse:	En person, der ønsker at registrere madvarer, følge udløbsdatoer, få opskriftsforslag og holde sit køleskab opdateret. Systemet skal være hurtigt og intuitivt, da brugeren ikke forventes at have teknisk erfaring. Forventet teknisk niveau: lavt.

Aktør Navn:	Husstandsmedlem
Type:	Primær aktør
Beskrivelse:	En inviteret eller godkendt bruger, som har adgang til et delt køleskab. Medlemmet kan se og administrere de samme varer som de øvrige godkendte medlemmer. Forventet teknisk niveau: lavt.

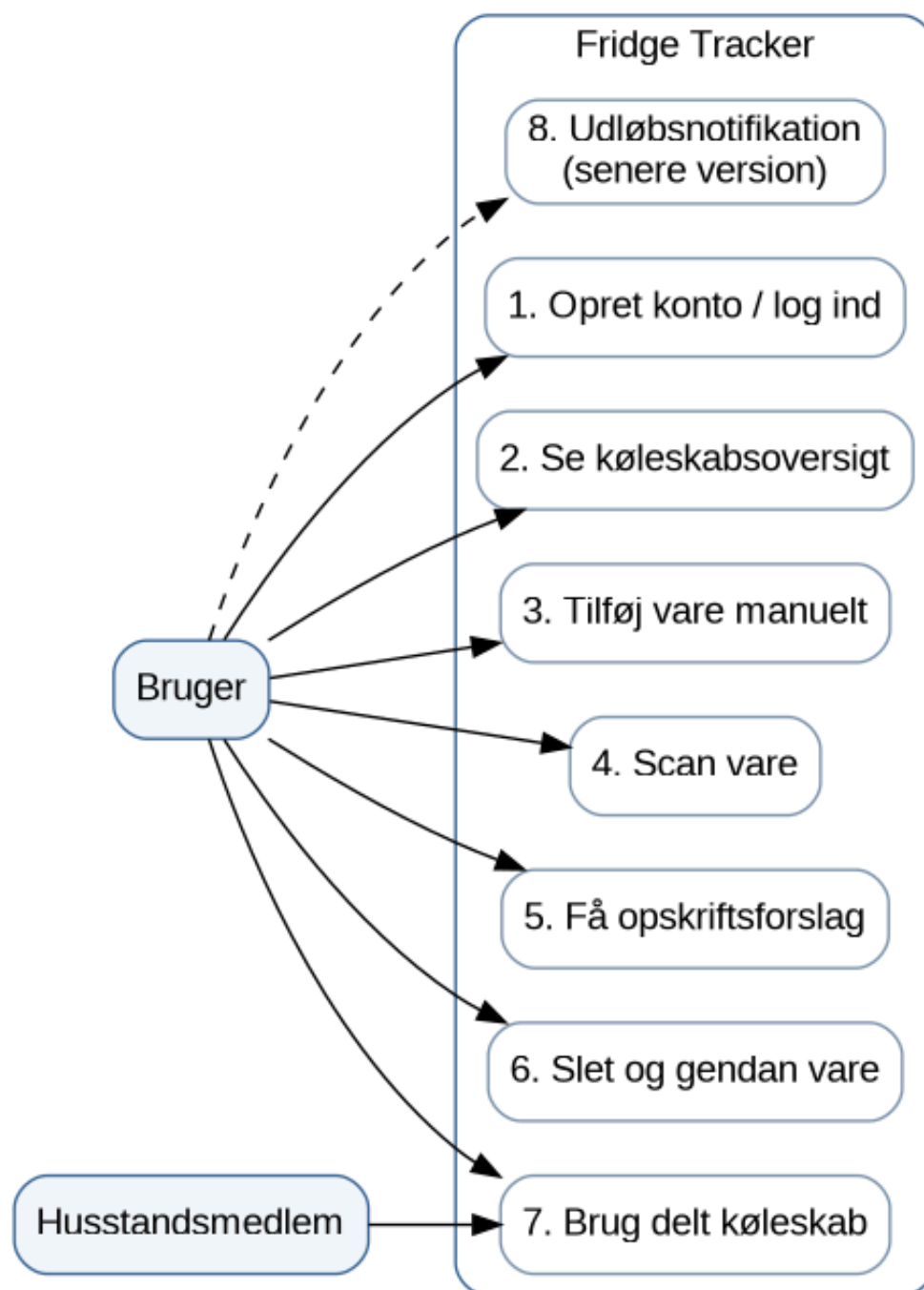
Aktør Navn:	Firebase
Type:	Sekundær aktør / eksternt system
Beskrivelse:	Firebase håndterer autentificering, realtidssynkronisering og lagring af bruger-, husstands- og varedata. Firestore security rules afgør, hvilke data brugeren må læse og skrive. Forventet teknisk niveau: højt.

Aktør Navn:	Eksterne API'er
Type:	Sekundær aktør / eksternt system
Beskrivelse:	Open Food Facts leverer produktdata ved stregkodeopslag, mens opskrifts-API'et leverer opskriftsforslag og detaljer. Forventet teknisk niveau: højt.

3.4 Use case-diagram

Figur 3 illustrerer brugerens interaktion med de use cases, der beskrives i afsnit 4.1.

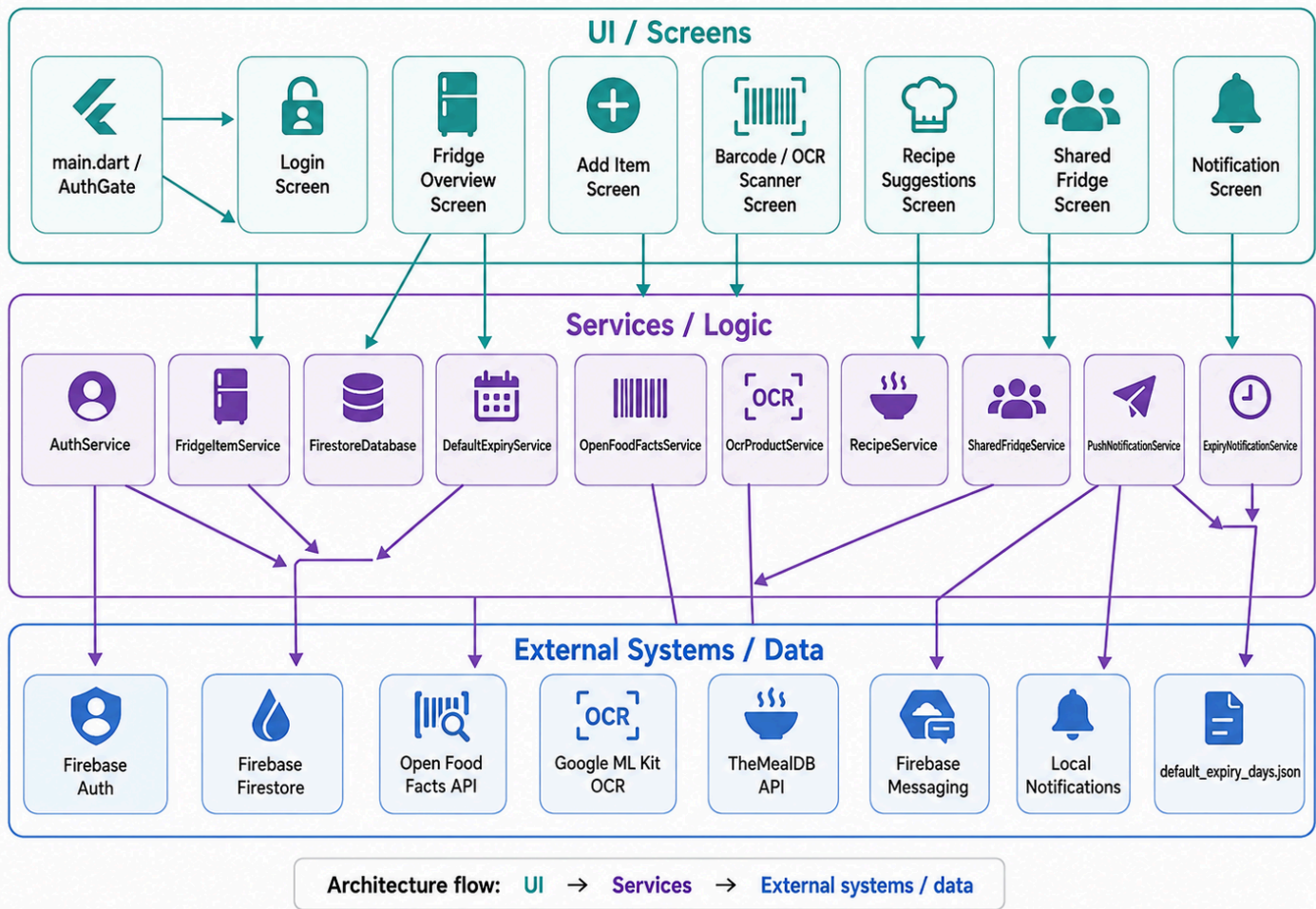
Figur 3 - Use case-diagram



3.5 Hardware/software-miljø

Systemet er udviklet som en cross-platform mobilapplikation i Flutter. Appen kommunikerer med Firebase og eksterne API'er over internetforbindelse.

Fridge Tracker – Component Diagram (Code Architecture)



Benyttet software:

- Flutter og Dart
- Firebase Authentication
- Cloud Firestore
- Firebase Cloud Messaging / lokale notifikationer (planlagt)
- Google ML Kit
- Open Food Facts API
- Eksternt opskrifts-API
- Git og GitHub
- Android Studio / Visual Studio Code

Benyttet hardware:

- Android- eller iOS-enhed med kamera
- Udviklingscomputer med Flutter SDK
- Internetforbindelse til Firebase og eksterne API'er

3.6 Sikkerhed

Brugerens login håndteres af Firebase Authentication. Adgangskoder gemmes ikke direkte af appen. Kommunikation med Firebase og eksterne API'er foregår via HTTPS.

Firestore security rules skal sikre, at brugeren kun kan læse og skrive egne data samt data fra delte køleskabe, hvor brugeren er et godkendt medlem. Varedata valideres ud fra forventede felter og datatyper.

3.7 Testsituationer og acceptkriterier

Systemet testes gennem automatiserede tests og praktiske brugertests. Testene skal blandt andet kontrollere login, validering, realtidsopdatering, farvekoder, manuel tilføjelse, scanning, standardudløbsdato, sletning med fortryd, opskrifter og adgang til delte køleskabe.

Den nuværende version betragtes som godkendt, når de aktive acceptkriterier er opfyldt. Notifikationskriterierne er markeret som fremtidige og indgår ikke i godkendelsen af denne version.

4 FUNKTIONELLE KRAV

4.1 Use Cases

PA = Primær aktør, SA = Sekundær aktør

4.1.1 Opret konto og log ind

ID og Navn:	1. Opret konto		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	Firebase Authentication
Mål:	PA opretter en konto og får adgang til sit køleskab.		
Initiering:	PA åbner appen og vælger opret konto.		
Startbetingelser:	Appen er installeret, og der er internetforbindelse til Firebase Authentication.		
Slutbetingelser:	PA er autentificeret og sendes til køleskabsoversigten.		
Normalforløb:	1. PA indtaster email og adgangskode. 2. Systemet validerer oplysningerne. 3. Firebase opretter eller godkender brugeren. 4. Appen åbner køleskabsoversigten.		
Alternativt forløb:	Eksisterende bruger vælger login i stedet for oprettelse.		
Undtagelser:	Ugyldig email, for kort adgangskode, email allerede i brug eller forkerte loginoplysninger.		
Prioritet:	Høj		
Frekvens:	Ved første oprettelse.		
Firmapolitik:	Adgangskoden skal være mindst 6 tegn.		
Andet:			
Antagelser:	PA har en gyldig emailadresse.		

ID og Navn:	1. Log ind		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	Firebase Authentication
Mål:	PA logger ind og får adgang til sit køleskab.		
Initiering:	PA åbner appen og vælger login.		
Startbetingelser:	Appen er installeret, og der er internetforbindelse til Firebase Authentication.		
Slutbetingelser:	PA er autentificeret og sendes til køleskabsoversigten.		
Normalforløb:	1. PA indtaster email og adgangskode. 2. Systemet validerer oplysningerne. 3. Firebase opretter eller godkender brugeren. 4. Appen åbner køleskabsoversigten.		
Alternativt forløb:			
Undtagelser:	Ugyldig email, for kort adgangskode, email allerede i brug eller forkerte loginoplysninger.		
Prioritet:	Høj		
Frekvens:	Ved login.		
Firmapolitik:			
Andet:			
Antagelser:	PA har en gyldig emailadresse.		

4.1.2 Se køleskabsoversigt

ID og Navn:	2. Se køleskabsoversigt		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	Cloud Firestore
Mål:	PA får et aktuelt overblik over varer og udløbsstatus.		
Initiering:	PA logger ind eller åbner oversigten.		
Startbetingelser:	PA er logget ind.		
Slutbetingelser:	Alle tilgængelige varer vises og opdateres løbende.		
Normalforløb:	1. Appen starter en Firestore-stream. 2. Varerne hentes og sorteres efter udløbsdato. 3. Navn, kategori og udløbsdato vises. 4. Varer markeres rødt ved <=3 dage, orange ved 4-7 dage og grønt ved >7 dage.		
Alternativt forløb:	Hvis køleskabet er tomt, vises en venlig tom-tilstand.		
Undtagelser:	Ved netværksfejl kan tidligere cachede Firestore-data vises, hvis de findes lokalt.		
Prioritet:	Høj		
Frekvens:	Meget ofte		
Firmapolitik:			
Andet:			
Antagelser:	PA har adgang til det valgte personlige eller delte køleskab.		

4.1.3 Tilføj vare manuelt

ID og Navn:	3. Tilføj vare manuelt		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	Cloud Firestore
Mål:	PA tilføjer en vare ved at udfylde en formular.		
Initiering:	PA trykker på manuel tilføjelse.		
Startbetingelser:	PA er logget ind.		
Slutbetingelser:	Varen gemmes og vises i oversigten.		
Normalforløb:	1. PA indtaster navn. 2. PA vælger kategori. 3. PA vælger udløbsdato. 4. Systemet validerer felterne. 5. Varen gemmes i Firestore.		
Alternativt forløb:	PA annullerer og vender tilbage til oversigten.		
Undtagelser:	Obligatoriske felter mangler eller data kan ikke gemmes.		
Prioritet:	Høj		
Frekvens:	Ofte		
Firmapolitik:			
Andet:			
Antagelser:	PA kender varens navn og udløbsdato.		

4.1.4 Scan og tilføj vare

ID og Navn:	4. Scan og tilføj vare		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	ML Kit / Open Food Facts
Mål:	PA registrerer en vare hurtigere ved hjælp af kameraet.		
Initiering:	PA trykker på scan-funktionen.		
Startbetingelser:	PA er logget ind, og appen har kameraadgang.		
Slutbetingelser:	Varen gemmes med tilgængelige produktdata og en foreslået udløbsdato.		
Normalforløb:	1. Kameraet åbnes. 2. PA scanner tekst eller en stregkode. 3. Appen genkender produktnavn eller henter produktdata. 4. Appen finder standardudløb i den lokale JSON-database. 5. Varen gemmes i Firestore.		
Alternativt forløb:	Hvis scanning ikke giver et resultat, fortsætter PA med manuel tilføjelse.		
Undtagelser:	Kameraadgang afvises, stregkoden findes ikke, eller API'et er utilgængeligt.		
Prioritet:	Høj		
Frekvens:	Ofte		
Firmapolitik:			
Andet:			
Antagelser:	Produktet har læsbar tekst eller stregkode.		

4.1.5 Få opskriftsforslag

ID og Navn:	5. Få opskriftsforslag		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	Opskrifts-API
Mål:	PA får inspiration til opskrifter baseret på køleskabets varer.		
Initiering:	PA åbner siden Opskrifter.		
Startbetingelser:	PA er logget ind og har varer i køleskabet.		
Slutbetingelser:	En liste med relevante opskrifter vises, og PA kan åbne detaljer.		
Normalforløb:	1. Appen udvælger relevante varer. 2. Appen sender søgeord til opskrifts-API'et. 3. Resultater vises. 4. PA åbner en opskrift. 5. Ingredienser og fremgangsmåde vises.		
Alternativt forløb:	Hvis ingen opskrifter findes, vises en tom-tilstand.		
Undtagelser:	Ingen internetforbindelse eller API-fejl.		
Prioritet:	Medium		
Frekvens:	Jævnligt		
Firmapolitik:			
Andet:			
Antagelser:	API'et understøtter mindst ét af søgeordene.		

4.1.6 Slet og gendan vare

ID og Navn:	6. Slet og gendan vare		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	Cloud Firestore
Mål:	PA holder køleskabsoversigten opdateret.		
Initiering:	PA anvender slettefunktionen på en vare.		
Startbetingelser:	PA er logget ind og har mindst én vare.		
Slutbetingelser:	Varen slettes eller gendannes, hvis PA vælger Fortryd inden 5 sekunder.		
Normalforløb:	1. PA sletter varen. 2. Varen fjernes fra Firestore. 3. Appen viser en Fortryd-handling i 5 sekunder.		
Alternativt forløb:	PA trykker Fortryd, og varen gendannes.		
Undtagelser:	Gendannelse mislykkes på grund af databasefejl.		
Prioritet:	Høj		
Frekvens:	Ofte		
Firmapolitik:			
Andet:			
Antagelser:	PA har skriverettigheder til varen.		

4.1.7 Brug delt køleskab

ID og Navn:	7. Brug delt køleskab		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger / Husstandsmedlem	Sekundær Aktør:	Cloud Firestore
Mål:	Flere godkendte brugere deler samme køleskabsoversigt.		
Initiering:	PA åbner eller tilslutter sig en husstand.		
Startbetingelser:	PA er logget ind og er godkendt medlem.		
Slutbetingelser:	PA kan se og administrere husstandens varer.		
Normalforløb:	1. Husstanden åbnes. 2. Firestore kontrollerer medlemskab. 3. Fælles varer vises. 4. Ændringer synkroniseres til øvrige medlemmer.		
Alternativt forløb:	PA kan fortsætte med sit personlige køleskab.		
Undtagelser:	Ikke-medlem forsøger adgang og afvises af security rules.		
Prioritet:	Medium		
Frekvens:	Ofte i delte husstande		
Firmapolitik:			
Andet:			
Antagelser:	Husstanden er oprettet, og medlemslisten er korrekt.		

4.1.8 Udløbsnotifikation - senere version

ID og Navn:	8. Udløbsnotifikation - senere version		
Lavet af:	Andrei, Azad og Dylan	Dato:	12/06-2026
Primær Aktør:	Bruger	Sekundær Aktør:	Notifikationservice
Mål:	PA bliver gjort opmærksom på varer, der nærmer sig udløb.		
Initiering:	En vare kommer inden for den valgte advarselsperiode.		
Startbetingelser:	Funktionen er implementeret, og PA har tilladt notifikationer.		
Slutbetingelser:	En notifikation vises med varens navn og relativ udløbsinformation.		
Normalforløb:	1. Appen identificerer en vare tæt på udløb. 2. En notifikation oprettes, f.eks. "Mælk udløber om 2 dage". 3. PA trykker på notifikationen. 4. Fridge Tracker åbnes.		
Alternativt forløb:	Hvis notifikationer er deaktiveret, vises ingen besked.		
Undtagelser:	Tilladelse mangler eller planlægning fejler.		
Prioritet:	Fremtidig		
Frekvens:	Efter behov		
Firmapolitik:			
Andet:	Dette use case er ikke et godkendelseskrav for den nuværende version.		
Antagelser:	Telefonens operativsystem tillader notifikationer.		

4.2 Krav om funktionalitet

Krav-ID	Funktion	Krav
FK01	Login	Opret konto, log ind og log ud med inputvalidering.
FK02	Køleskabsoversigt	Vis varer med navn, kategori og udløbsdato samt farvekoder: rød <=3 dage, orange 4-7 dage og grøn >7 dage.
FK03	Manuel tilføjelse	Tilføj vare via formular med validering af obligatoriske felter.
FK04	Slet vare	Slet vare og tilbyd Fortryd i 5 sekunder.
FK05	Notifikation - senere	Udløbsnotifikationer implementeres i en senere version.
FK06	Scan vare	Understøt OCR og strejkode med ML Kit og Open Food Facts.
FK07	Standardudløb	Foreslå udløbsdato fra lokal JSON-database.
FK08	Opskrifter	Vis opskriftsforslag og detaljer baseret på køleskabsvarer.
FK09	Delt køleskab	Understøt fælles køleskab for godkendte husstandsmedlemmer.

5 IKKE-FUNKTIONELLE KRAV

Krav-ID	Kategori	Testbart ikke-funktionelt krav	Krav-IDTestmetode / godkendelseskriterium
IFK01	Sikkerhed	Brugeren skal være logget ind med Firebase Authentication for at kunne læse eller ændre data. Firestore-reglerne skal afvise adgang til andre brugeres køleskabe, medmindre køleskabet er delt med brugeren. Al kommunikation med Firebase og eksterne API'er skal foregå via HTTPS.	Test med en bruger uden login og en bruger uden delingsadgang. Alle ikke-godkendte læse og skriveforsøg skal afvises. En godkendt delt bruger skal kunne tilgå det delte køleskab.
IFK02	Performance og data	Appen skal vise login-skærmen eller køleskabsoversigten inden for 3 sekunder efter opstart på en normal testtelefon. En køleskabsoversigt med op til 100 varer skal indlæses inden for 2 sekunder ved normal internetforbindelse. Firestore-opdateringer skal vises i appen inden for 2 sekunder .	Mål tiden fra appens start til første brugbare skærm. Test oversigten med 100 varer. Tilføj eller redigér en vare i Firestore, og mål tiden til ændringen vises i appen.
IFK03	Brugervenlighed	Brugeren skal kunne åbne formularen til tilføjelse af en vare med højst 2 tryk fra køleskabsoversigten. Ugyldige eller manglende formularfelter skal vise en forståelig fejlbesked. En tom køleskabsoversigt skal vise en tydelig tom-tilstand. Varer skal farvekodes som rød ved 3 dage eller mindre, orange ved 4–7 dage og grøn ved mere end 7 dage .	Gennemfør en brugertest, hvor mindst 4 ud af 5 testpersoner kan tilføje en vare uden hjælp. Kontrollér fejlbeskeder, tom-tilstand og farvekoder med varer med forskellige udløbsdatoer.
IFK04	Skalerbarhed	Et køleskab skal kunne indeholde mindst 500 varer og deles med mindst 10 brugere uden nedbrud eller tab af data. Oversigten skal fortsat kunne indlæses inden for 3 sekunder ved 500 varer.	Opret testdata med 500 varer og 10 delte brugere. Kontrollér, at data kan læses, oprettes og redigeres korrekt, og mål indlæsningstiden.
IFK05	Platformskompatibilitet	Appen skal kunne bygges og køre på Android 10 eller nyere og iOS 15 eller nyere . Login, visning af varer, manuel tilføjelse, scanning og håndtering af kamera-tilladelser skal fungere på begge platforme.	Byg og start appen på mindst én Android-enhed eller emulator og én iOS-enhed eller simulator. Gennemfør de centrale brugerflows uden kritiske fejl. Afvis og godkend kamera-tilladelsen for at teste begge situationer.

IFK06	Vedligeholdelse og test	Koden skal være opdelt i skærme, services og datamodeller. Alle kritiske funktioner, herunder login, varehåndtering, udløbsdatoer og datalagring, skal være dækket af automatiserede tests. Alle tests skal bestå før en version kan godkendes. Servicelaget skal have mindst 70 % testdækning .	Kør projektets automatiserede testpakke og generér en coverage-rapport. Der må være 0 fejlede tests , og servicelagets testdækning skal være mindst 70 %.
-------	-------------------------	---	--

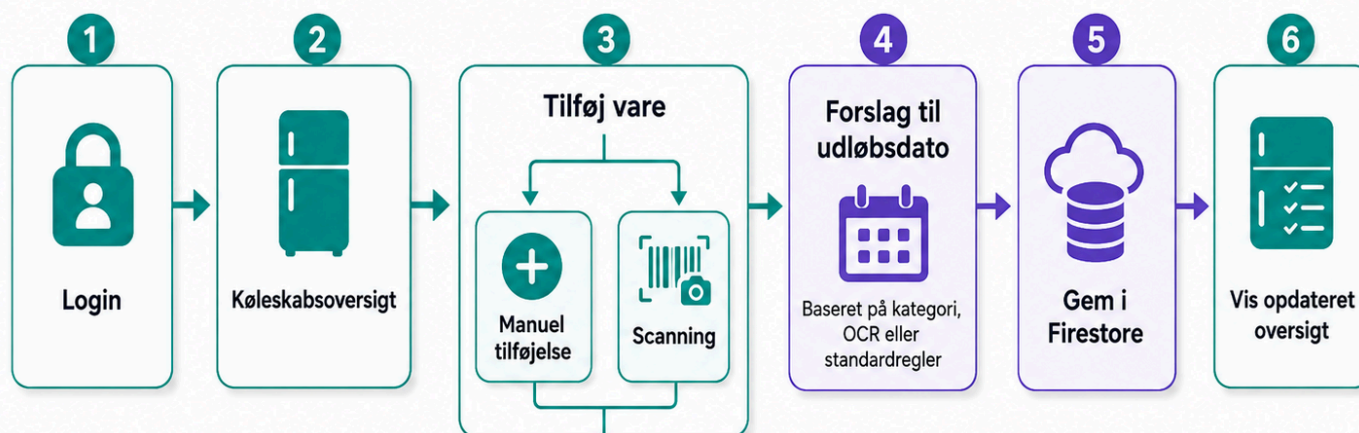
6 DESIGNKRAV

6.1 Brugerfladekrav

- Designet skal være enkelt og let at forstå uden instruktion.
- De vigtigste funktioner skal kunne nås fra køleskabsoversigten.
- Udløbsstatus skal vises tydeligt med rød, orange og grøn markering.
- Formularer skal vise fejl tæt på det felt, der skal rettes.
- Knapper og tekst skal have en størrelse, der fungerer på almindelige mobilskærme.
- Tom- og fejltilstande skal forklare, hvad brugeren kan gøre bagefter.

6.2 Visuelt flow

Fridge Tracker – Visuelt flow



Normalt flow: login → oversigt → tilføjelse/scanning → udløbsdato → gem → opdateret oversigt

7 DOKUMENTHISTORIK

Version	Dato	Ændring	Forfatter
0.1	18/05-2026	Første version af kravspecifikationen.	Projektgruppen
0.2	12/06-2026	Farvekoder ændret til rød ≤3 dage, orange 4-7 dage og grøn >7 dage.	Andrei
0.3	12/06-2026	Notifikationer markeret som senere version. Use cases og ikke-funktionelle krav tilpasset den aktuelle app.	Andrei
1.0	12/06-2026	Dokumentet omstruktureret efter lærerens eksempel med diagrammer og detaljerede use case-tabeller.	Projektgruppen

8 BILAG

8.1 Definitioner og ordforklaringer

Begreb	Forklaring
API	En grænseflade, som gør det muligt for systemer at udveksle data.
Firebase Authentication	Tjeneste til sikker brugeroprettelse og login.
Cloud Firestore	Cloud-database med realtidsopdatering og lokal cache.
Flutter	Framework til cross-platform apps skrevet i Dart.
ML Kit	Googles bibliotek til blandt andet tekst- og strekcodegenkendelse.
OCR	Optical Character Recognition - genkendelse af tekst i billeder.
Open Food Facts	Offentlig produktdatabase, som kan slås op via strekcode.
Security rules	Regler, der bestemmer hvem der må læse og skrive Firestore-data.
Use case	Beskrivelse af hvordan en aktør bruger systemet til at opnå et mål.

8.2 Datamodel

Figur 4 viser en forenklet datamodel for brugere, husstande, køleskabe og varer. Den præcise Firestore-struktur kan variere, men relationerne viser den logiske opdeling.

Figur 4 - Forenklet datamodel

